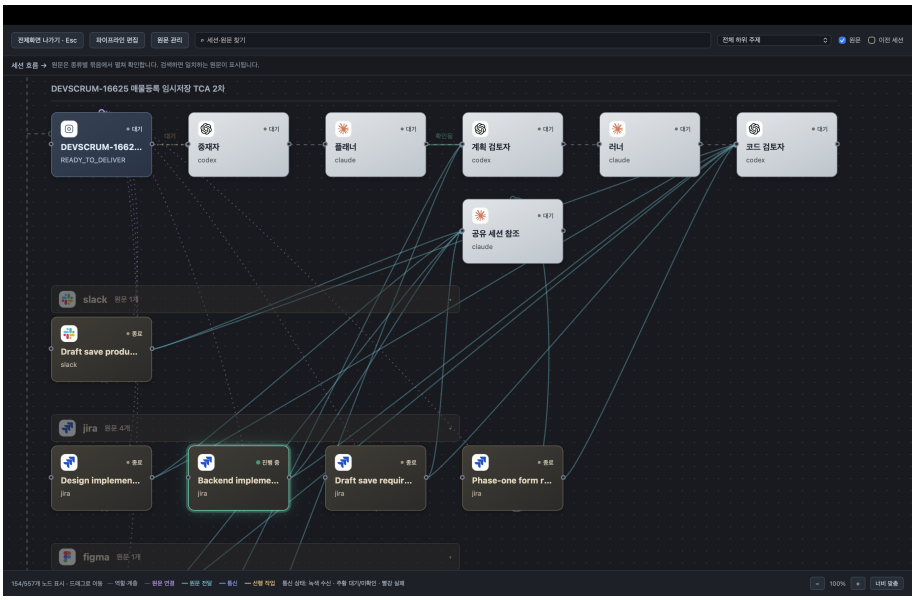
# Claude/Codex 공동 개발 작업 자동화

같은 대화에서 계획/구현/검토를 이어 가고, 실행과 결과를 기록하는 도구.




## 이 프로젝트를 시작한 계기

한 에이전트에 계획과 구현을 맡겼을 때 필요한 작업이 빠지거나, 잘못된 전제로 진행이 막히고 일부 현상을 과하게 해석하는 일을 겪었다. Claude와 Codex의 의견을 직접 주고받아 계획을 다듬자, 서로 놓친 문제를 보완해 합의한 계획의 완성도가 높아졌다.

이 방식을 계속 활용하고 싶었지만, 매번 응답을 전달하고 합의를 진행하기는 어려웠다. 그래서 계획 합의 → 구현 → 교차 검토 → 검증을 자동으로 이어 가는 Consensus Room을 만들었다. 작업 범위와 우선순위, 설계 판단과 계획 승인은 직접 맡는다.

세션 그래프: 역할별 참여자와 외부 원문 연결. 화면의 배정과 상태는 촬영 당시 기준.

# Code Agent의 주관적 성능 비교

실사용에서 느낀 주관적인 Code Agent의 성능 비교표.

계획/검증과 코드 작성에서 직접 느낀 주관적인 평가다. 동일 조건으로 측정한 벤치마크 결과는 아니다.

| Code Agent | 계획 / 검증 | 코드 작성 / 토큰 효율 |
|---|---|---|
| GPT-6 Astra | Claude 모델보다 우수 | 코드 작성 시 토큰 소비가 매우 큼 |
| GPT-6.1 Sol | 전반적으로 열세 | 전반적으로 열세 |
| Claude Fable 5.1 | Astra보다 열세 | - |
| Claude Opus 5.5 | Astra보다 열세 | 토큰 소비 대비 코드 작성 성능이 압도적으로 우수 |

## 시스템 개요

리팩토링을 빠짐없이 마무리하는 플랜모드와, 여러 원문의 요구사항 변화를 추적하며 구현하는 티켓모드를 둔다. 외부 자료의 변경은 자동으로 수집해 작업 세션에 전달하고, 엔진은 실행 조건과 결과 기록을 맡는다.

## 실제 업무에 적용한 결과

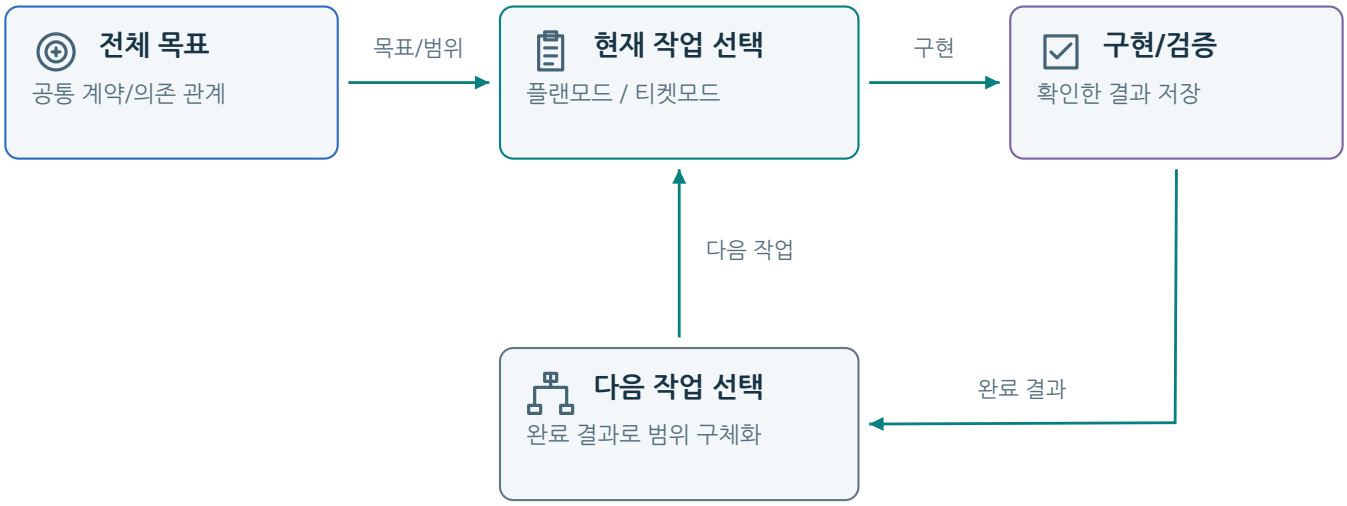
10월 개편 전 엔진으로 운영 iOS 앱의 Swift 6 전환을 약 4주에 완료했다. 공통 모듈부터 앱과 테스트까지 단계별로 검증해 9월 23일 개발 브랜치에 통합했고, CI 검사로 Swift 5 설정의 재유입도 막았다.

사용자의 직접 사용 경험에 따른 평가 / 2026.10

# 플랜모드와 티켓모드의 선택 기준

플랜모드는 리팩토링의 완결에, 티켓모드는 요구사항 변경을 추적하는 구현에 권장.

초기에는 모든 작업을 계획부터 시작해, 요구사항이 바뀔 때마다 계획 검토를 되풀이했다. 완결성이 중요한 리팩토링은 목표 구조와 순서를 먼저 합의하고, 스프린트 업무는 여러 원문의 변경을 추적하며 구현하도록 두 모드로 나눴다.




## 1. 플랜모드: 완결성이 중요한 리팩토링

시간이 오래 걸리더라도 정한 범위를 빠짐없이 마무리하는 것이 중요한 작업에 권장한다. Swift 6 리팩토링, 모듈화, 아키텍처 변경이 대표적이다. 서비스 원문의 실시간 변경을 따라가는 업무보다 목표 구조와 완료 조건을 먼저 합의할 작업에 적합하다.

단계별 의존 관계를 정하고, 두 모델이 검토한 계획을 내가 승인한 뒤 구현/교차 검토/검증을 잇는다. 여러 Markdown 파일의 스냅샷과 변경분을 보존하고, 현재 계획과 승인한 판이 같은지 확인한다.

## 2. 티켓모드: 요구사항 변경을 추적하며 구현

Jira/Slack/Figma/백엔드 문서 등 여러 곳에서 요구사항이 실시간으로 바뀌어, 변경 내역을 추적하면서 구현해야 하는 서비스 작업에 권장한다. 스프린트 작업이 대표적이다. 원문 변경을 수집해 세션에 전달하고, 내가 우선순위와 반영 범위를 정한다.

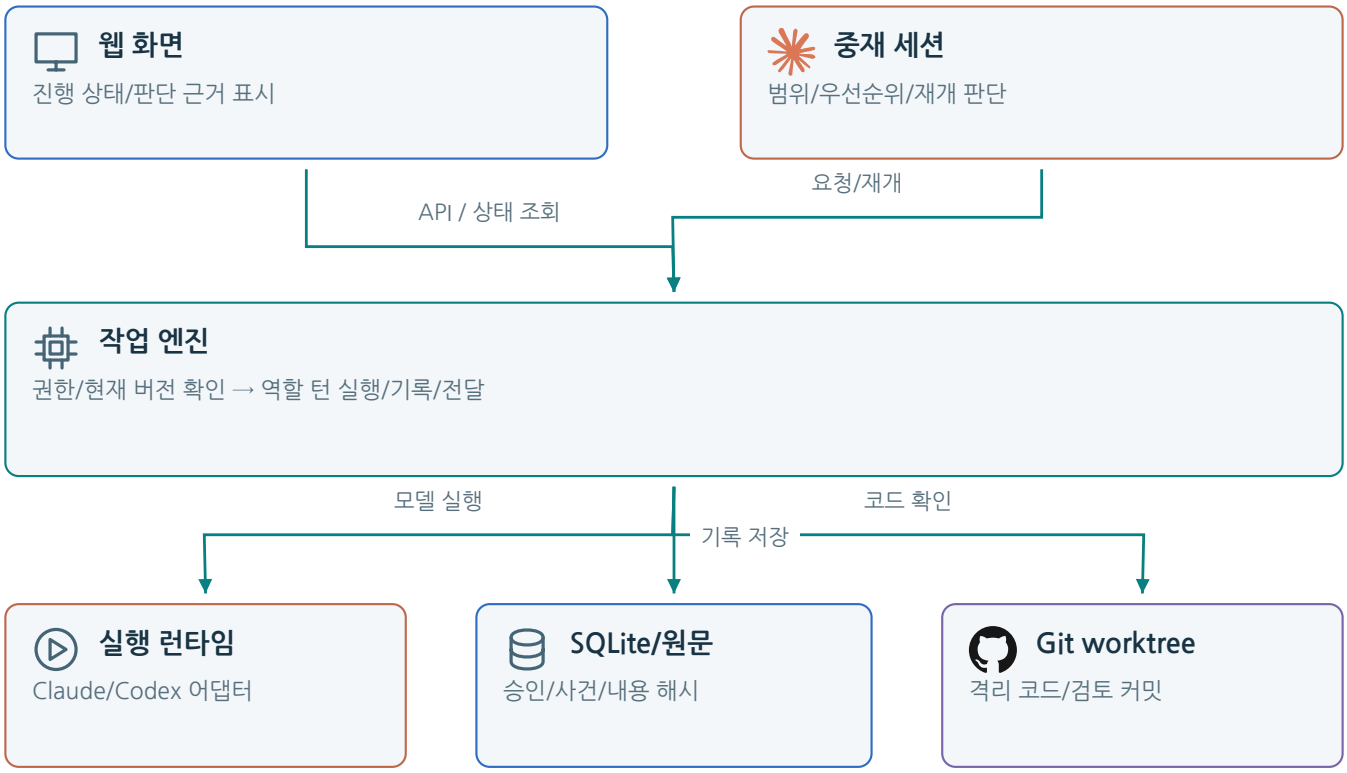
두 모델의 계획 합의를 필수로 두지 않아 변경마다 전체 계획을 다시 합의하지 않는다. 구현 뒤 코드 리뷰와 검증은 유지한다. 내부에서는 플랜모드를 계획 작업(planned), 티켓모드를 일반 작업(ticket)으로 구분한다.

src/server/planBundle.ts / src/server/engine/planning.ts / src/shared/prompts.ts

# 화면/엔진/실행 도구의 역할

작업자가 내용을 판단하고, 엔진은 실행 조건/중복 실행/결과 보존을 책임진다.

React/Vite는 상태를 보여 주고, Node.js/TypeScript/Fastify 서버는 허용된 행동을 실행한다. SQLite는 승인/배정/사건/중단 위치를 보존한다. 엔진이 지적의 의미까지 여러 곳에서 판정하던 구조를 줄이고, 역할별 응답을 다음 세션에 전달하도록 책임을 좁혔다.




## 역할별 구현 책임

React/Vite는 화면을, Node.js/TypeScript/Fastify는 서버를 구성한다. 공통 실행 런타임은 Claude/Codex CLI의 옵션과 출력 차이를 어댑터 뒤로 모은다. Git 작업 트리는 코드 변경을 격리하며, 원문과 결과 파일은 내용 해시로 식별한다.

## 공통 실행과 정책의 경계

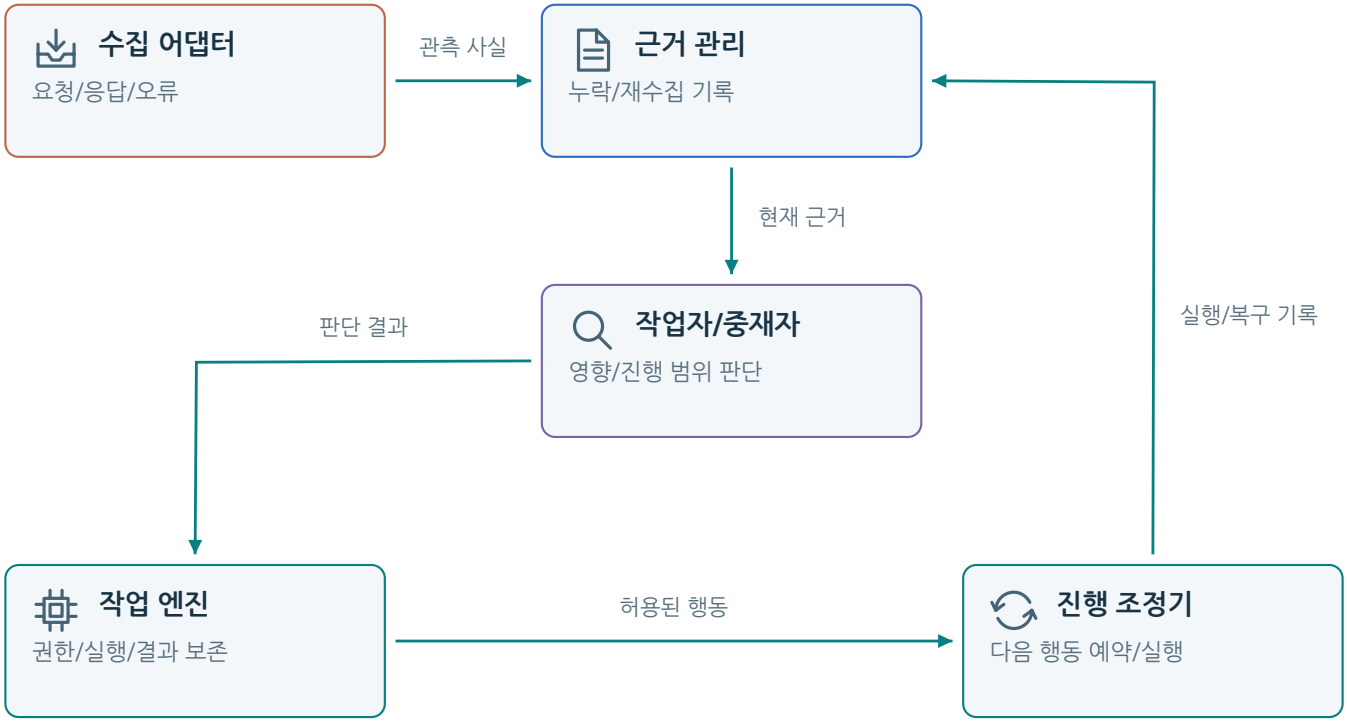
엔진과 호스트 리뷰는 같은 실행 계약을 쓰지만 각자의 승인과 예산을 소유한다. 런타임이 운영 DB를 대신 수정하거나 리뷰 승인을 만들어 내지 않는다. 이렇게 나누면 공급자 실행 방식을 고쳐도 작업 승인 규칙까지 바꾸지 않아도 된다.

src/server/runtime/ / src/server/engine/ / src/server/database.ts

# 계층별 책임과 공통 계약

수집 사실과 작업 판단을 분리하고, 다음 행동은 엔진의 공통 조건으로 실행.

같은 원문 변경을 수집기, 계획 게이트, 구현 게이트가 각각 해석했다. 한 곳의 정지를 풀어도 다른 곳이 다시 막았다. 복구 코드를 덧붙일수록 어느 계층의 판단이 정본인지 추적해야 하는 경로도 늘었다.




## 수집부터 실행까지의 책임

수집기는 원문/변경/조회 실패를 기록한다. 작업자와 중재자는 그 사실로 진행 범위를 판단한다. 엔진은 현재 실행의 소유권, 승인 조건, 결과 형식을 검사하고 다음 역할에 응답을 보낸다. 원문 변경만으로 계획과 세션을 초기화하지 않는다.

## 공통 처리로 모으고 옛 판단 경로를 삭제

같은 원인을 여러 곳에서 다르게 분류하던 쟁점 판정, 확인 턴, 재작성 원장과 옛 계획 루프를 제거했다. 실행 호출과 결과 보존은 공통 경계로 모았다. 다만 권한, Git HEAD, 실행 신원처럼 서로 다른 위험을 막는 검사는 유지했다.

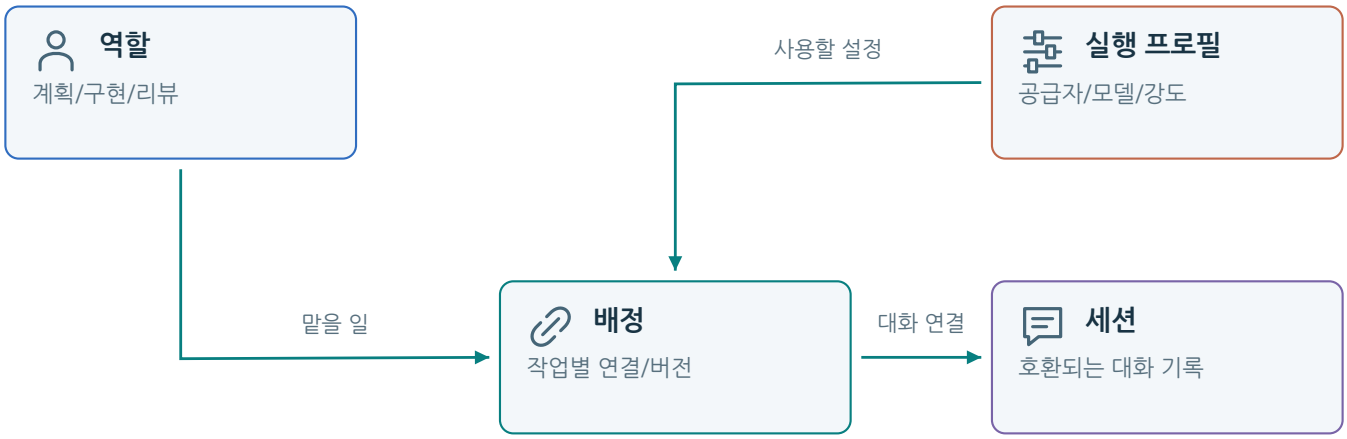
삭제 순서도 어려웠다. 옛 소비처가 남은 상태에서 복구 코드를 먼저 지우면 재개가 막혔다. 새 흐름을 먼저 공급하고 호출부를 바꾼 뒤, 남은 소비처와 검사를 함께 정리했다. 이름 검색만으로 간접 소비를 놓친 사례도 있었다.

src/server/engine/core.ts / src/server/engine/turnExecutor.ts / src/server/runtime/invoke.ts

# 역할/프로필/배정/세션

역할은 책임, 프로필은 실행 설정, 배정은 연결 관계, 세션은 대화 기록.

설계자/구현자/검토자라는 역할은 허용할 행동과 결과 형식을 설명한다. 실행 프로필은 공급자/모델/추론 강도/공급자별 옵션을 담는다. 배정은 어느 작업에서 어떤 프로필을 쓰는지와 그 버전을 연결한다. 세션은 해당 배정의 대화 기록이다.




## 배정 변경과 호환성

이 구분이 없으면 중재자를 Claude에서 Codex로 바꿀 때 역할과 무관한 설정까지 다시 만들어야 한다. 개편에서는 배정과 기능 지원 여부를 먼저 확인한 뒤 공통 실행 요청으로 변환한다. 전역 배정을 작업별 배정으로 덮어쓸 수 있지만, 기록 없이 설정을 바꾸지는 않는다.

## 새 공급자의 연결 조건

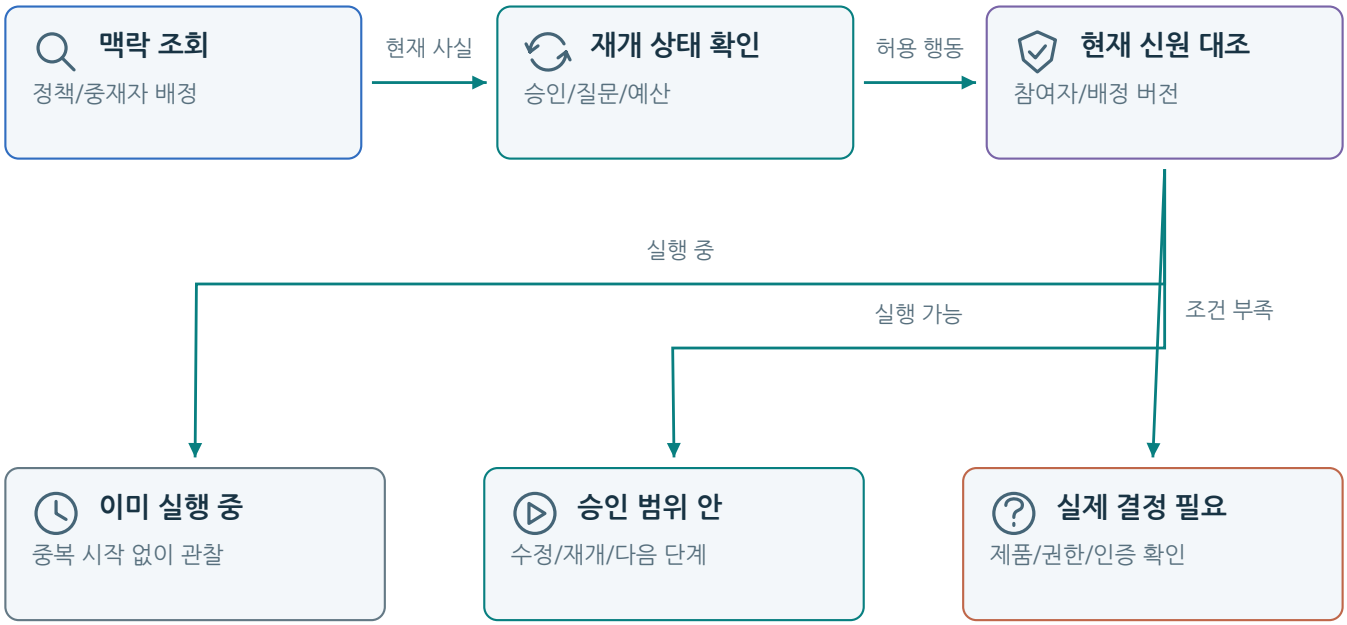
실제 어댑터는 Claude와 Codex 두 개다. 임의의 AI 도구를 연결하면 곧바로 동작하는 구조는 아니다. 새 공급자는 실행/권한/세션/사용량/결과 파싱 계약을 구현해야 한다. ultracode 같은 옵션도 공급자별 계약으로 남겨 공통 추론 강도와 혼동하지 않는다.

src/shared/roles.ts / src/server/roleAssignments.ts / src/server/runtime/providers.ts

# 중재자의 확인 순서

대화의 마지막 문장보다 서버의 현재 상태/허용 행동/배정 신원을 먼저 확인.

중재자는 어떤 작업이 진행 중인지, 무엇 때문에 멈췄는지, 남은 질문과 예산이 무엇인지 확인한다. 이미 승인된 범위 안의 수정과 다음 준비된 단계를 조율하고, 서버에 저장된 자동 진행 상태를 확인한다. 제품 결정/인증/권한/예산처럼 실제로 부족한 조건은 이유를 남기고 요청한다.




## 중재자 교체와 상태 확인

대화의 마지막 문장만 보고 재개하지 않도록 현재 맥락 조회, 재개 상태 확인, 역할 배정 신원 확인 순서를 둔다. 이전 승인과 질문을 유지하고 이미 실행 중인 작업을 중복 시작하지 않게 하는 절차다.

## 하위 작업의 연결 누락과 복구

상위 작업에는 중재 세션이 연결돼 있었지만 새 하위 작업은 미배정으로 표시됐다. 배정 조회가 자기 작업과 전역 설정만 확인해 상위 연결을 놓친 것이 원인이었다. 가장 가까운 상위 중재자 배정을 조회하되 원래 버전과 세션을 유지하도록 고쳤다.

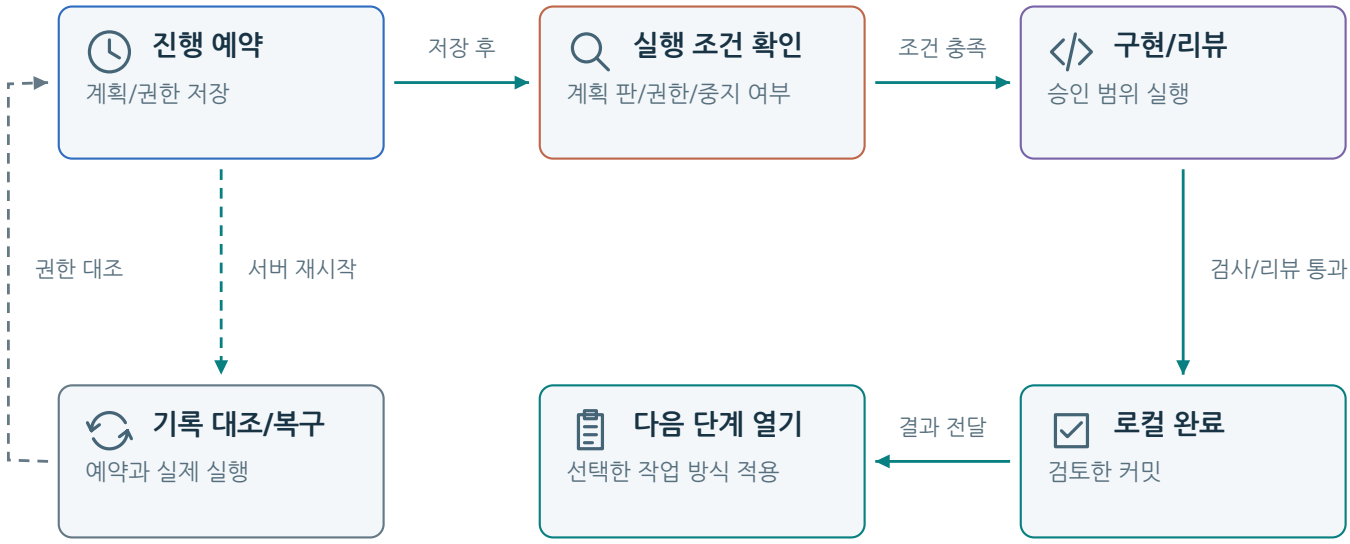
배정을 새로 만들면 기존 신원과 승인 기록이 어긋날 수 있어 조회 규칙을 수정했다. 하위 작업에 명시한 배정은 우선한다. 공개 API에서 두 단계 상속, 서버 재시작 뒤 연결 유지, 하위 재배정 뒤 이전 신원의 거부를 확인했다.

docs/mediation/policy.md / src/server/mediationAutonomy.ts

# 서버가 맡는 연속 실행

역할 응답으로 다음 턴을 정하고, 승인된 자동 진행 범위만 서버에 맡긴다.

계획 합의가 끝나도 별도 근거 심사, 마무리 확인, 양쪽 확인 응답을 기다리면 구현은 시작되지 않았다. 먼저 누락된 심사 호출을 연결했지만, 한 단계를 고칠수록 다른 복구 조건이 드러났다. 결국 모델이 내용의 합의를 다시 확인하는 사슬을 없앴다.




## 승인한 범위의 자동 진행

계획 작성/감사/개정을 같은 역할 세션으로 잇고, 현재 판에 동의하면 사용자 승인을 기다린다. 구현 뒤에는 코드 리뷰를 받고, 수정 요청은 같은 구현 세션으로 돌아간다. 자동 진행 예약에는 승인한 계획과 허용한 전달 범위를 기록한다.

## 다음 턴 전 한 번 보류

중재자가 계획 왕복 중 보류를 요청하면 실행 중인 턴은 끝까지 두고 결과를 저장한다. 다음 계획 턴으로 넘어가기 직전에 한 번 멈추며, 결정문을 실어 재개하면 저장한 다음 지점에서 같은 세션으로 잇는다. 개정 횟수만으로 자동 정지하거나 계획 승인을 생략하지 않는다.

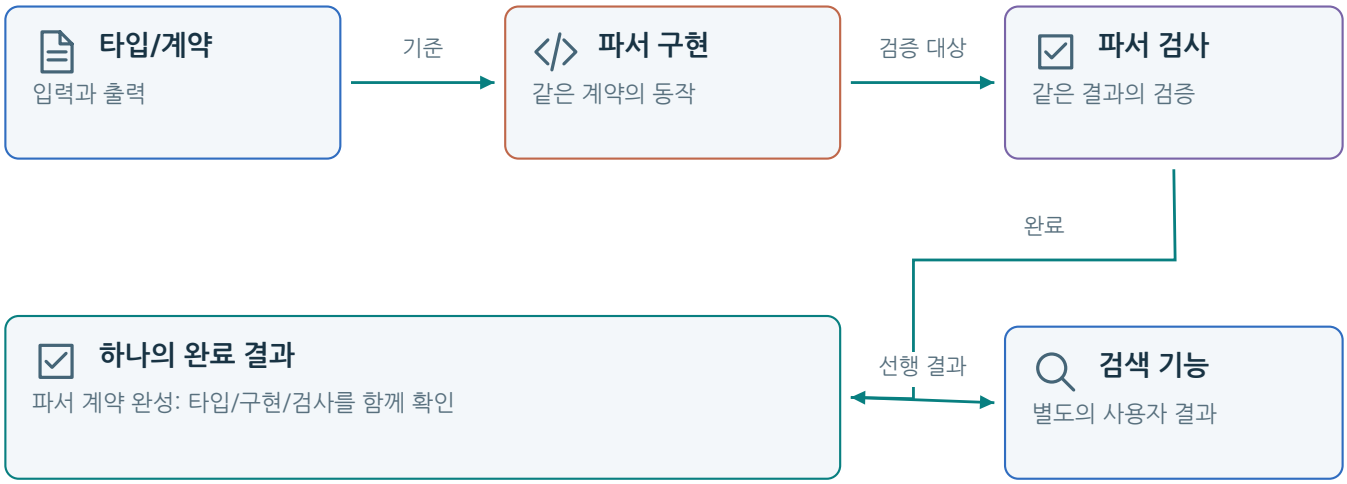
서버 재시작 뒤에는 기존 예약과 실제 실행을 대조한다. 유효한 자동 진행은 복구하지만 사용자 중지나 취소된 권한은 되살리지 않는다. 새 계획은 다시 승인받고, 다음 티켓 작업은 열어 둔 뒤 구현 시작을 기다린다.

src/server/engine/continuation.ts / src/server/continuationStore.ts / src/shared/workflowLifecycle.ts

# 완료 결과를 기준으로 한 단계 분리

같은 결과를 만드는 설계/구현/검사는 한 단계 안의 체크리스트로 구성.

설계/구현/테스트를 각각 별도 단계로 만들거나 파일별로 단계를 나누면 같은 내용을 여러 번 설명하고 검토하게 된다. 개편에서는 한 결과를 완성하는 작업을 단계 안의 체크리스트로 묶는다. “이 단계를 끝내면 무엇을 확인할 수 있는가?”를 먼저 적는다.




## 단계 분리의 판단 기준

분리는 앞 단계의 결과가 있어야 판단할 수 있는 경우, 독립적인 검증이 필요한 경우, 실패 시 따로 되돌릴 필요가 있는 경우로 설명한다. 단계 수나 코드 줄 수를 일률적으로 제한하지 않는다.

## 현재 단계의 완료 조건

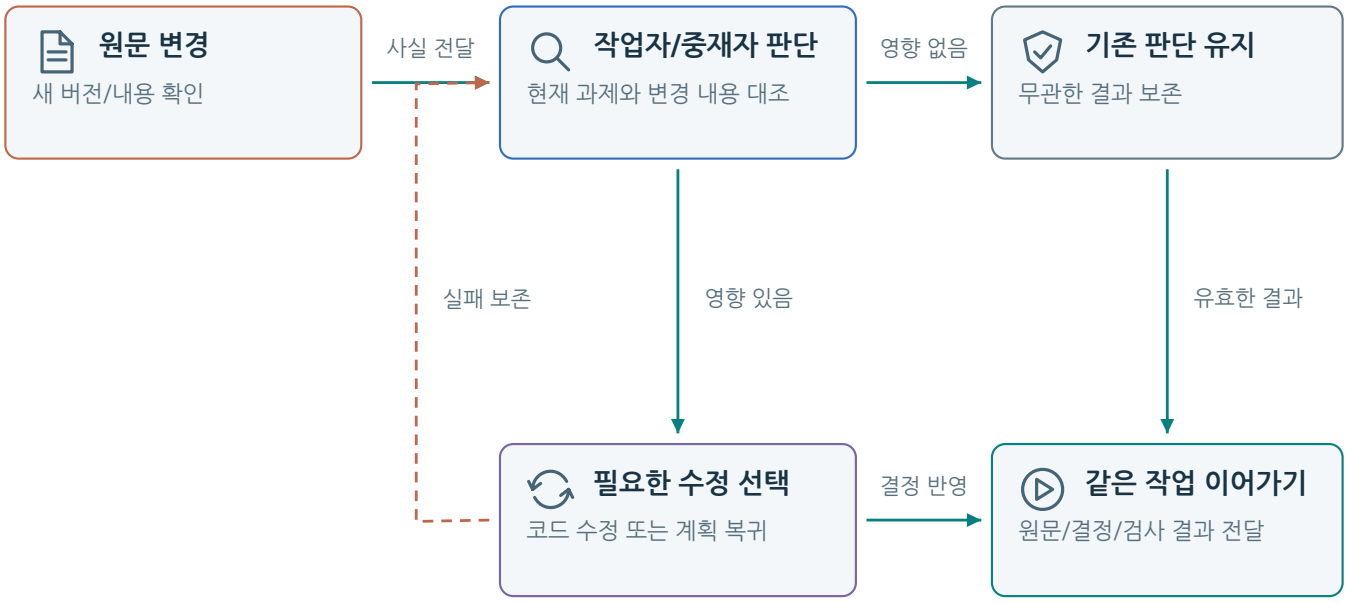
현재 단계에는 완료 조건, 검사 방법, 예산, 해결해야 할 질문이 필요하다. 미래 단계는 개요로 남길 수 있다. 기본적으로 한 단계를 실행하며, 외부 정보나 사용자 결정을 기다릴 때만 의존하지 않는 준비된 단계로 진행할 수 있다. 예산 소진을 기다림으로 바꿔 다른 실행을 시작하지 않는다.

src/shared/workGroups.ts / src/server/workGroups.ts

# 근거 변경과 판단 보존

원문 변경은 새 사실로 전달하고, 기존 계획/세션/작업 트리는 보존한다.

원문이 바뀔 때마다 계획 승인과 세션을 무효화하면, 구현한 코드가 있어도 다시 계획으로 돌아갔다. 영향 심사를 따로 붙여 해결하려 했지만 심사 실패를 복구하는 경로까지 필요해졌다. 현재는 바뀐 사실을 전달하고, 영향 판단은 작업자와 중재자가 맡는다.




## 변경 사실과 초기화 요청의 구분

새 원문, 선택 변경, 조회 실패를 사건으로 남기고 각 세션에 다음 턴의 사실로 전달한다. 그 사실만으로 계획과 작업 트리를 초기화하지 않는다. 작업 중 받은 결정이 이미 시작한 턴에 소급해 들어가지는 않는다.

작업 범위 자체를 새로 시작하는 요청은 별개다. 현재 범위 세대와 명시적인 초기화 확인을 함께 받아야 한다. 검증 범위를 조정하는 결정은 일반 결정으로 전달하며, 범위 초기화에는 공식 되돌리기 동작이 없다.

## 재사용할 결과와 다시 확인할 결과

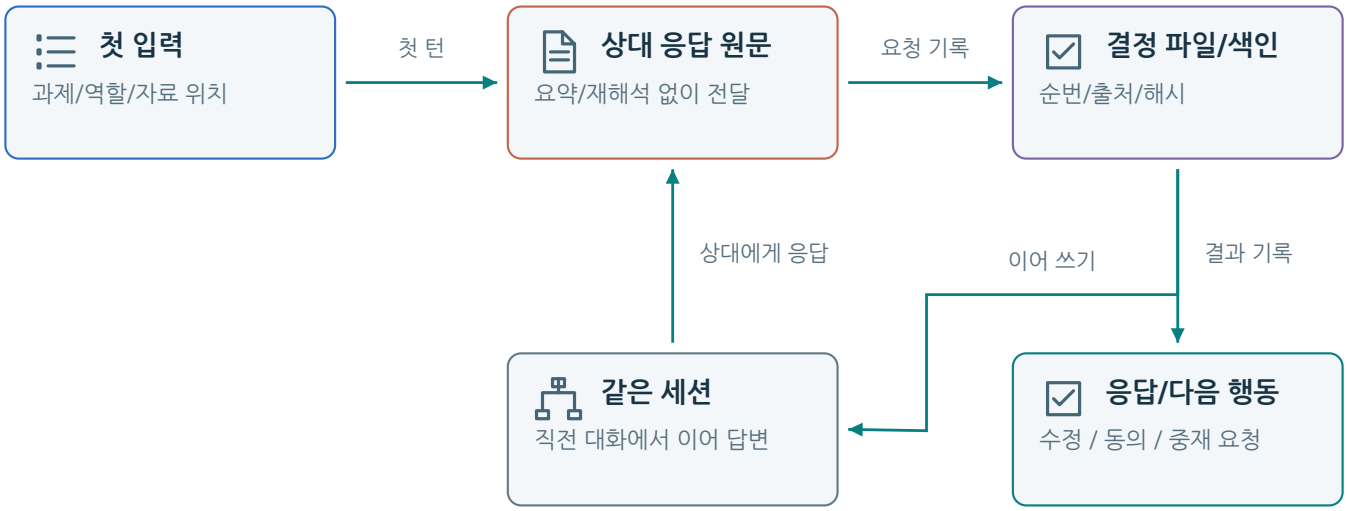
계획 수정이 필요하면 이유를 적어 같은 플래너에게 돌려보낸다. 작업 방식을 바꿔도 사건/사용량/작업 트리를 통째로 지우지 않는다. 중재자는 코드/도구/검사 입력을 대조해 유지할 결과와 다시 확인할 범위를 결정에 남긴다.

src/shared/turnContract.ts / src/shared/prompts.ts / src/server/workflow.ts

# 원문 전달과 대화의 연속성

상대 응답은 대화로 잇고, 긴 결정은 압축 뒤에도 다시 열 수 있는 파일로 보존.

옛 계획 제어에서는 요청서와 타임라인을 서버가 조각으로 나눠 읽게 했다. 한 계획 개정은 2,559초가 걸렸고, 64,434바이트 요청서를 처음부터 다시 읽은 기록이 남았다. 분할 크기와 재사용 규칙을 고쳐도 대화를 매번 다시 구성하는 부담은 남았다.




## 압축 뒤에도 다시 읽을 수 있는 결정

첫 턴에는 과제/지침/근거 위치를, 다음 턴에는 상대 응답 원문과 새 사실을 전달한다. 현재 범위 세대의 사용자 결정은 원문 파일과 색인으로도 보존한다. 파일 경로/판/해시와 결정별 순번/출처를 이어지는 턴에도 싣는다.

4,000자를 넘는 고유 결정은 파일 구간으로 안내하고, 같은 본문은 파일에 한 번만 둔다. 승계 결정도 원본 해시가 확인될 때만 같은 본문으로 묶는다. 저장된 사건과 결정의 순서는 바꾸지 않는다.

## 접근 경로와 실제 읽기의 구분

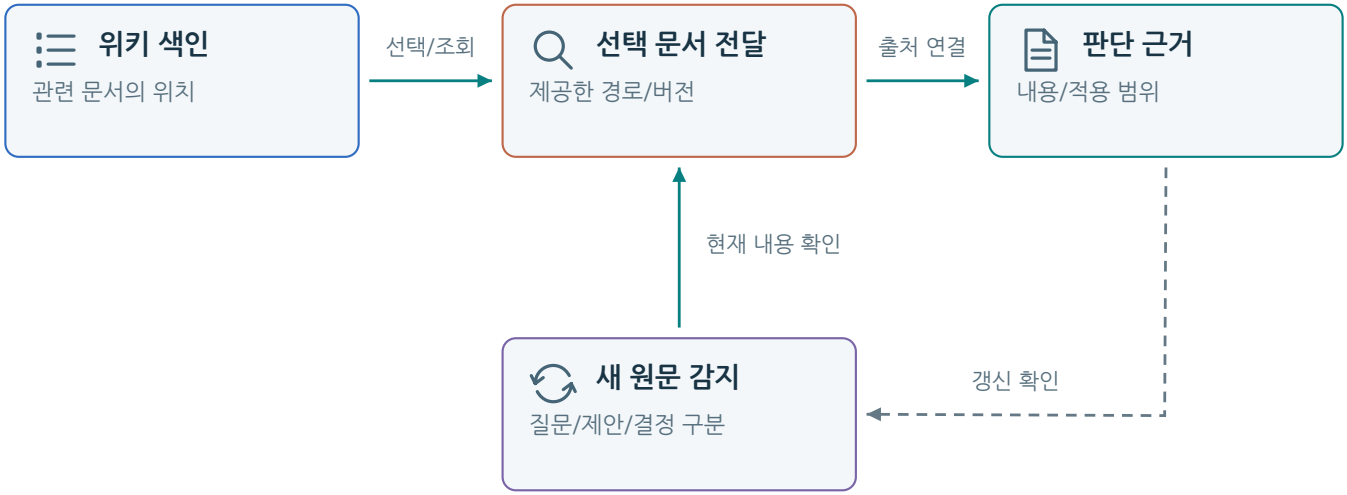
문맥 압축은 Claude/Codex가 처리한다. 엔진은 압축된 대화를 복원하거나 결정 준수를 검사하지 않는다. 대신 긴 원문을 다시 열 수 있게 한다. 세션 교체 때 사실이 다시 전달될 수 있으며, 이 변경의 실사용 토큰 절감률은 아직 같은 조건으로 측정하지 않았다.

src/shared/prompts.ts / src/server/engine/planning.ts / src/server/engine/delivery.ts

# 위키 검색과 실제 근거의 구분

기능 위키 선택, 문서 전달, 실제 근거 판단을 구분.

역할 지침과 실행 경로까지 검색어로 쓰자 기능 위키보다 무관한 문서가 선택됐다. 검색 입력을 토픽과 상위 토픽의 제목/목표로 분리하고, 기능이 명확하지 않은 제목은 현재 범위의 사용자 요청으로 보완했다.




## 기능 위키를 먼저 선택

라우터의 기능별 위키 안내에 연결된 문서를 기능 위키로 인식한다. 검색 주제마다 대표 문서를 먼저 고르며, 위키가 있으면 우선한다. 라우터와 추가 문서 최대 4개, 총 180,000바이트 한도는 유지한다. 빌드 같은 일반 기술어만으로 사용자 요청을 생략하지 않는다.

## 제공한 자료와 사용한 근거

새 세션에는 선택 문서 본문을, 재개에는 같은 검색 입력으로 만든 매니페스트를 보낸다. 기록은 무엇을 제공했는지 보여 주며 모델이 모두 읽고 따랐음을 증명하지는 않는다. 계획 턴에는 설정된 설계 철학 문서 한 개의 읽기 경로도 제공한다.

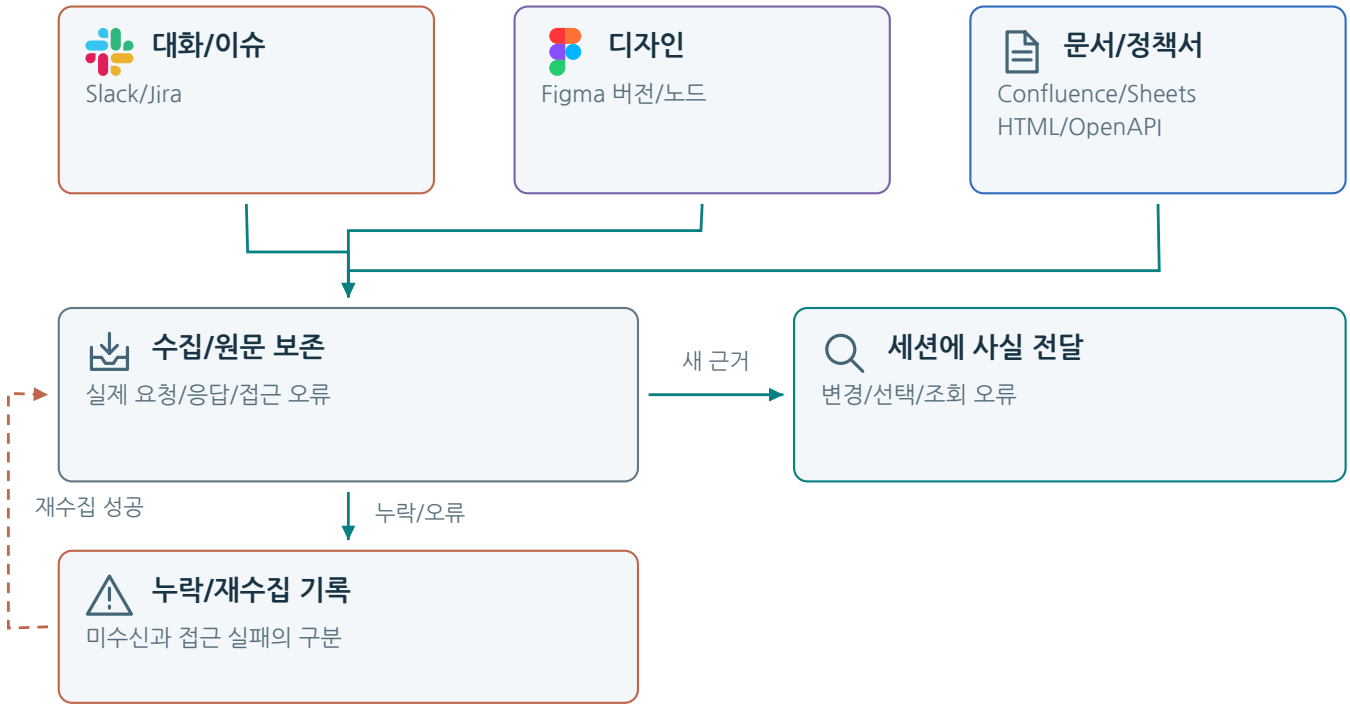
위키에 외부 근거 연결 블록이 있어야 원문 신선도를 대조한다. 본문 갱신과 근거 연결은 별도 운영 작업이다. 엔진은 허용된 갱신 제안의 경로/상태/해시를 확인하며 위키 폴더 전체의 쓰기를 허용하지 않는다.

src/server/memoryRetrieval.ts / projectMemory.ts / wikiEvidence.ts / engine/planning.ts

# 외부 자료 수집과 변경 전달

자동 수집은 관측 사실을 남기고, 작업 세션이 변경의 영향을 판단한다.

Slack/Jira/Figma/Confluence/Google Sheets와 일반 HTTPS 문서를 등록한다. REST 수집기는 기본 15분 간격으로 재확인하고, 시작할 때 밀린 조회도 처리한다. 특히 요구사항이 자주 바뀌는 티켓모드에서 최신 자료를 작업 세션에 전달하는 데 사용한다. 닫힌 작업에서만 쓰는 출처는 정기 조회에서 제외한다.




## 수집과 영향 판단의 분리

수집 자체는 모델을 호출하지 않는다. 일반 문서의 HTML/OpenAPI와 연결된 앱의 원문을 보존하고, 변경/조회 오류를 작업 세션에 전달한다. Figma section이 개요만 반환하면 프레임 상세를 읽은 것으로 보지 않는다. 디자인과 구현의 일치는 별도로 확인한다.

## 조회만으로 바뀌던 Jira 해시

같은 구조화 응답도 실행 환경의 로캘에 따라 키 순서가 달라졌다. 기존 저장본과 호환되는 en-US 순서로 고정하고, 읽는 계정의 조회 시각인 fields.lastViewed만 제외했다. development와 외부 링크는 남겨 실제 변경을 숨기지 않는다.

## 오류가 다른 수집으로 번지지 않도록 분리

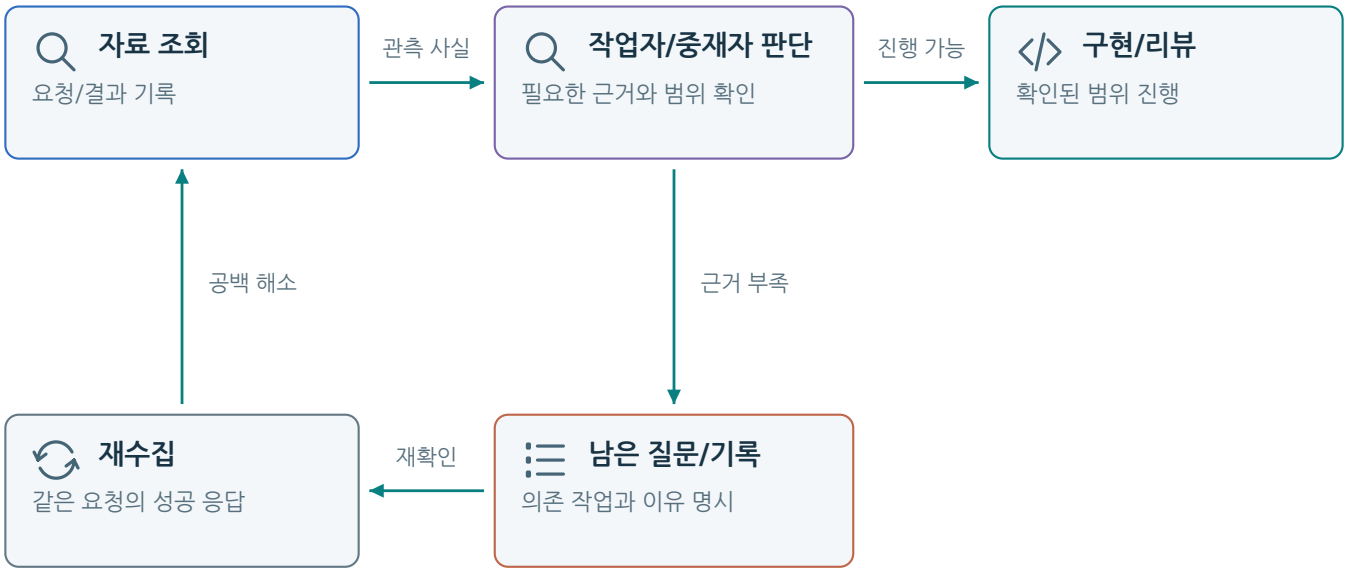
일반 도구 오류와 취소는 해당 호출의 실패로 남긴다. 시간 초과는 신원 세대를 올리지 않고 연결만 교체해 다음 호출을 받는다. 실제 계정 변경은 계속 확인하며, 실패한 호출을 즉시 자동 재시도하지 않는다. 저장된 원문의 존재와 최신 재확인 여부도 구분한다.

src/shared/externalEvidence.ts / src/server/evidence/discovery.ts / src/server/evidence/service.ts

# 자료 누락과 후속 작업

부족한 자료에 의존하는 부분만 후속 목록으로 넘기고, 확보한 근거의 범위에서 진행.

근거가 부족한 항목과 진행할 수 있는 항목을 나누려 했지만, 엔진이 공백을 쟁점으로 주입하고 별도 처분까지 요구하면서 판단이 겹쳤다. 구현자/리뷰어의 대화가 할 일을 정하도록 바꾼 뒤에는, 옛 자동 이연 정책도 함께 제거해야 했다.




## 한 자료의 실패가 전체 중지로 번진 사례

옛 재개 턴에서는 미수신 요청 36개 중 한 Figma 요청이 제한에 걸려 전체 실행이 멈췄다. 먼저 나머지 요청을 보존하고 가능한 일을 이어 가도록 고쳤다. 이후에는 엔진이 이연 쟁점을 합성하는 방식 자체가 필요한지 다시 검토했다.

## 관측 상태와 작업 처분의 분리

현재 봉투는 역할, 응답 원문, 다음 행동을 담는다. 엔진은 원문 공백을 별도 쟁점으로 만들지 않는다. 구현자나 리뷰어가 판단하기 어려우면 이유를 적어 중재를 요청하고, 중재자는 승인 범위에서 진행/보류를 정한다.

새 봉투가 표현할 수 없는 옛 To-do 필드를 요구하는 안내도 삭제했다. 작업 묶음의 과거 동결 결과에 남은 보류 기록은 출처와 원문을 함께 전달한다. 이는 이전 작업의 참고 기록이며, 새 지시나 자동으로 해결된 항목이 아니다.

src/shared/turnContract.ts / src/server/workGroups.ts / src/server/evidence/store.ts

# 계획/구현/리뷰의 세션 연결

호환되는 대화와 유효한 읽음 기록을 이어 사용하고, 검토자의 판단 맥락은 별도로 유지.

계획과 구현의 연속성을 살리려면 세션 ID만 보존해서는 부족했다. 이어지는 턴마다 전체 지침을 다시 붙이거나, 계획 변경으로 리뷰어 결속을 지우면 대화는 반복됐다. 현재는 역할별 첫 입력과 이어지는 응답을 구분한다.




## 대화를 이어 쓰는 조건

계획과 구현의 배정이 호환되고 연속성 정책이 켜져 있으면 계획 세션을 구현에 넘긴다. 코드 리뷰어도 같은 작업 범위에서 이어 쓴다. 계획 판을 바꾸거나 작업 방식을 전환했다는 이유만으로 리뷰어를 새로 만들지 않는다.

세션이 사라지면 실패를 남긴다. 다른 ID를 자동 채택하던 짝 복구 예외는 삭제했다. 새 세션이 필요하면 중재자가 역할과 사유를 명시해 교체한다. 이력은 남기며 작업 전체를 초기화하지 않는다.

## 검토자의 독립성과 재사용 범위

검토자는 고정된 계획과 코드 변경을 확인한다. 수정 후에는 같은 리뷰 대화에서 지적과 변경분을 확인한다. 형식 위반 교정도 같은 세션에서 한 번 수행하고, 여전히 유효하지 않으면 원문을 보존한 채 실패로 남긴다.

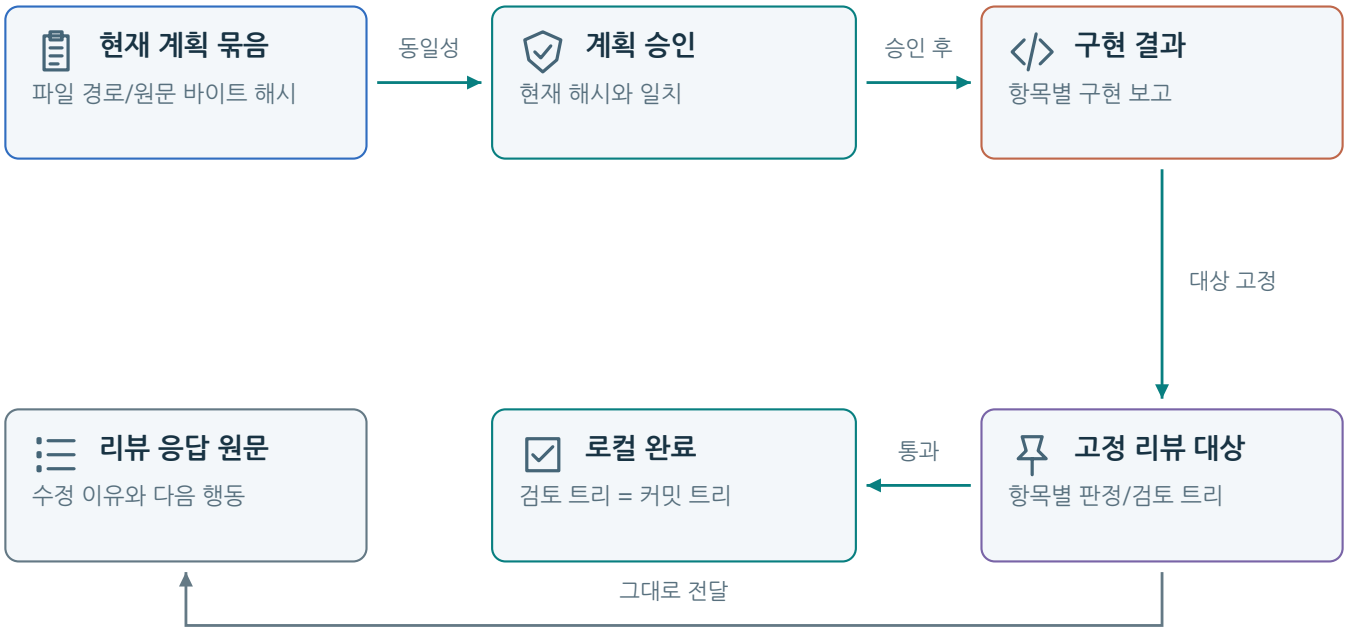
기존 세션을 이어 쓰는 것과 모든 입력의 중복을 없애는 것은 다르다. 인계 시 사실을 다시 받는 경우와 운영 비용은 별도로 확인해야 한다.

src/server/engine/core.ts / src/server/engine/turnExecutor.ts / src/server/planningStore.ts

# 승인 계획과 검토 코드의 동일성

계획 내용, 리뷰 대상, 실제 결과를 각각 고정하고 같은 대상인지 대조.

계획 작업은 여러 Markdown 파일을 한 묶음으로 확정한다. 상대 경로와 파일 바이트를 정해진 순서로 직렬화하고 SHA-256을 계산한다. 승인한 판과 현재 판이 같아야 구현한다. 일반 작업은 계획 해시를 구현 조건으로 요구하지 않는다.




## 계획 항목과 검토 대상의 연결

계획 작업은 승인 계획에 항목 ID/요구/완료 조건을 선언한다. 구현자는 각 항목의 구현 결과를, 코드 리뷰어는 충족 여부와 근거를 보고한다. 엔진은 선언한 항목의 누락과 보고 형식을 확인하고, 결과 기록을 승인 판/범위 세대/검토 트리에 연결한다.

목록이 없거나 형식이 잘못되면 모델 호출 전에 보완 경로를 안내한다. 이 검사는 모델이 쓴 근거의 진실성을 대신 판단하지 않는다. 티켓 작업에는 계획 항목 선언을 요구하지 않는다.

## 내용 해시와 코드 검토 범위

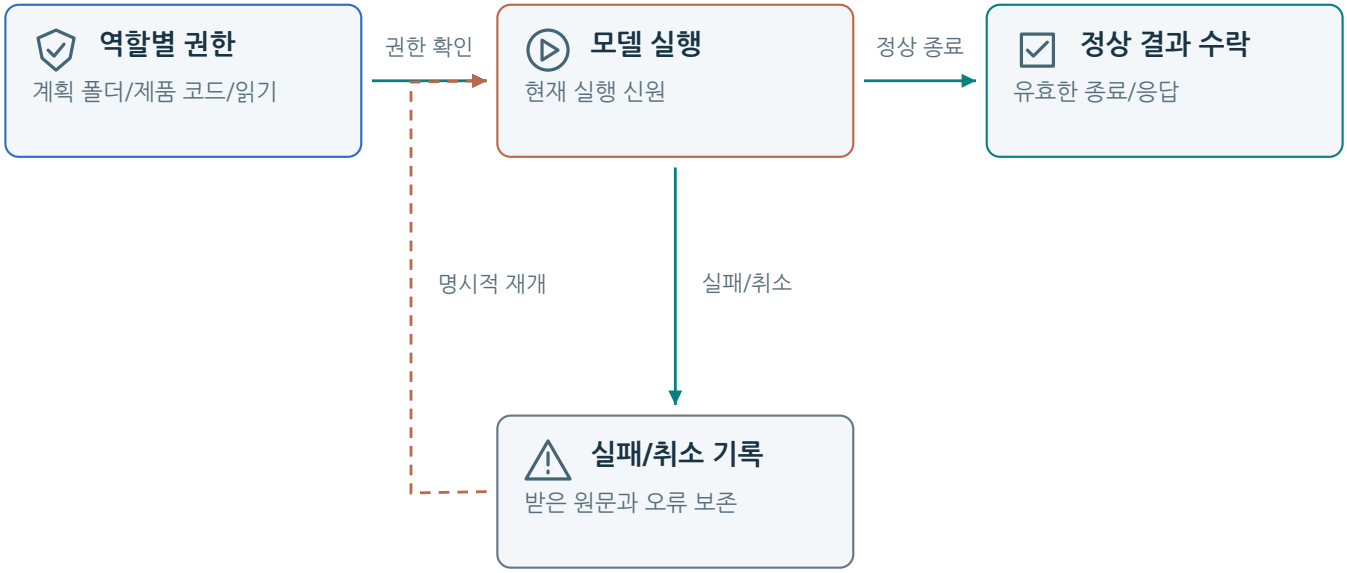
계획 버전과 코드의 검토 대상을 따로 고정한다. 검사나 리뷰 사이에 외부 커밋이 들어오면 다른 코드를 승인한 것으로 처리하지 않는다. 전달할 때도 리뷰한 트리와 같은지 대조한다. 계획 파일의 저장과 조회는 같은 원문 바이트 계약을 사용한다.

src/server/planBundle.ts / src/shared/planItems.ts / src/server/engine/planCoverage.ts

# 역할별 권한과 실행 종료

실행 권한/프로세스 종료/결과 형식을 모두 확인한 뒤 결과 수락.

계획 작성자는 지정된 계획 폴더를 쓰고, 검토자는 읽으며, 구현자는 허용된 제품 코드를 바꾼다. 보호할 도구 트리와 역할 권한은 유지한다. 확인을 위한 모델 턴을 없애도 실제 쓰기/실행 경계까지 없애지는 않는다.




## 실행 권한을 실제 도구 입력에 전달

계획 폴더/제품 코드/읽기 역할의 권한을 나누고 보호 도구 트리는 쓰기 대상에서 뺀다. 보호 목록과 안내는 같은 정본에서 만든다. Claude의 턴별 임시 폴더는 실제 경로 한 철자로 도구 규칙과 안내에 함께 싣는다.

## 합성 결과와 실제 턴 종료의 구분

Claude 재개 직후 이전 백그라운드 작업의 알림이 0턴 성공 결과로 먼저 나올 수 있다. 이를 최종 결과로 보던 정리 타이머가 실제 턴을 끊었다. 구조화 결과가 없는 합성 0턴 성공은 타이머 시작에서 빼고 실제 결과를 기다리도록 고쳤다.

## 받은 결과와 채택한 결과

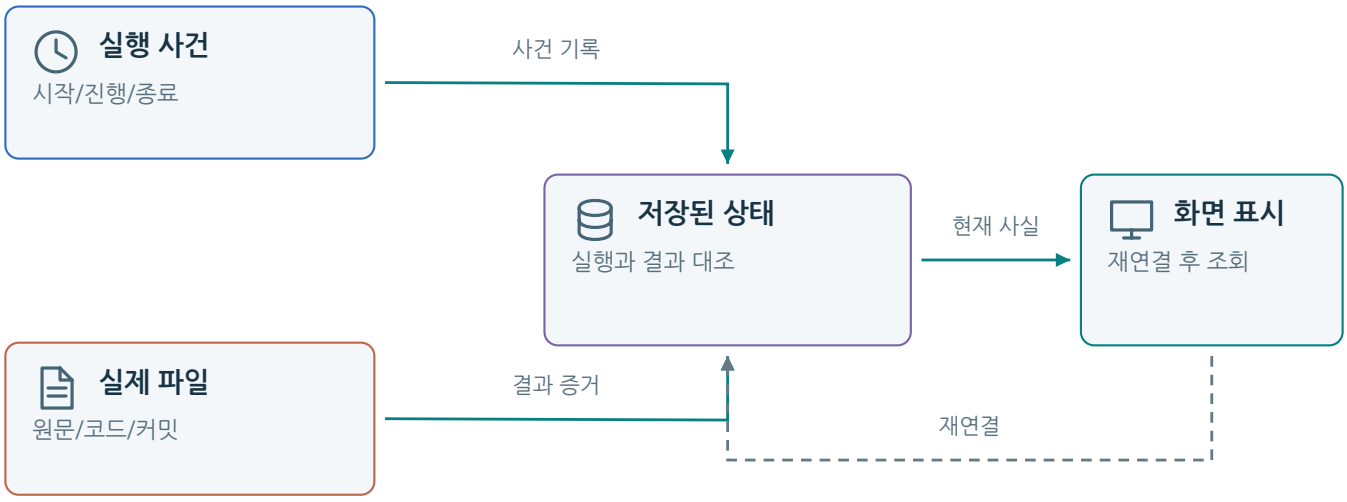
원문/세션/역할/요청 당시 입력 순번은 후처리 전에 보존한다. 취소/재계획 뒤 늦은 응답은 최신 상태를 덮어쓰지 못한다. 프로세스도 PID와 시작 신원을 확인해 정리한다. 결과 보존은 정상 완료나 무조건 재호출할 권한을 뜻하지 않는다.

src/server/adapters/claude.ts / src/server/adapters/codex.ts / src/server/engine/core.ts

# 진행 표시와 실제 작업의 구분

실행 시간/문장 증가/완료 선언을 서로 다른 증거로 확인.

계획/리뷰/구현 결과를 파일로 남기면 대화가 길어져도 필요한 원문을 다시 읽을 수 있다. 다만 별칭 파일이 다른 내용으로 교체될 수 있으므로 내용 해시로 고정한 자료를 기준으로 삼는다. DB의 참조와 실제 파일을 읽을 때 해시를 대조한다.




## 저장된 상태와 재연결

화면은 서버의 실행 기록과 이벤트를 받아 진행 상태를 보여 준다. 실행 시간이나 누적 문장이 늘어난 것만으로 진척을 판단하지 않는다. 현재 턴의 결과/실제 파일/읽은 자료를 대조하며, 재연결 뒤에도 저장된 상태를 확인한다.

## 검증 근거별 확인 범위

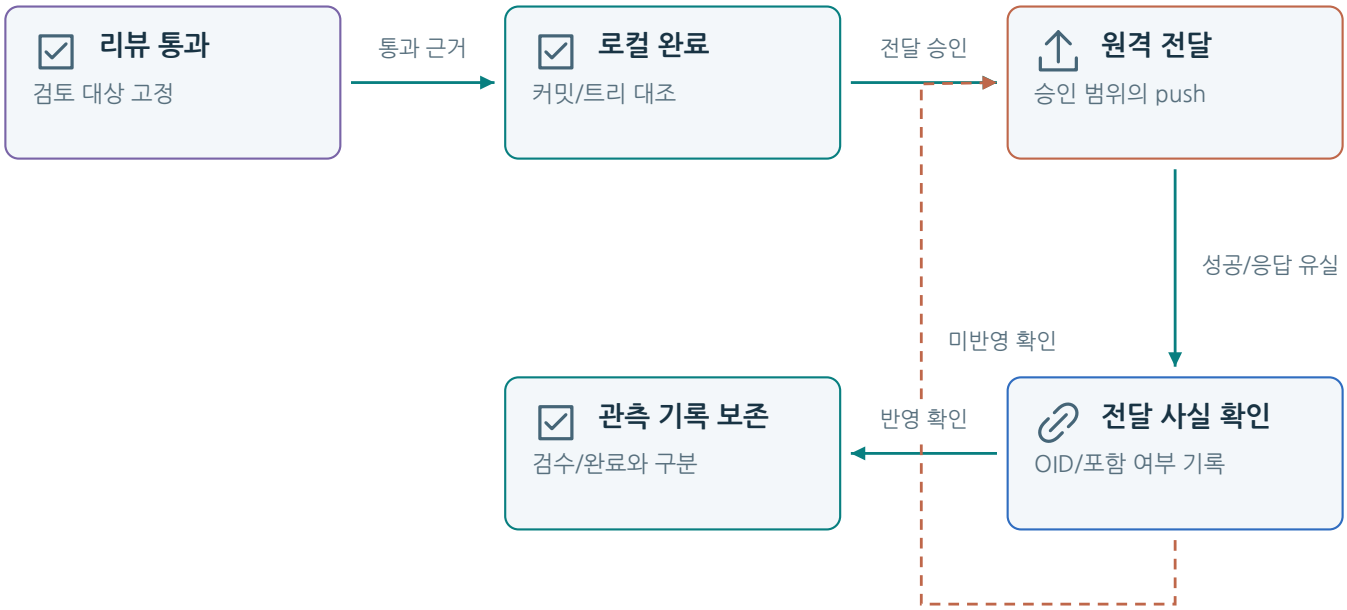
승인한 경로와 변경 규칙을 벗어났는지 확인할 수 있다. Swift 주석/문자열을 구분해야 하는 검사에서는 단순 문자열 치환으로 판단하면 오탐과 누락이 생긴다. 범위 규칙을 적용할 때도 해당 언어의 표현을 고려해야 한다.

서로 다른 증거를 하나의 성공 표시로 합치지 않는 것이 이 설계의 중요한 제한이다.

# 로컬 완료와 원격 전달

커밋, 푸시, 묶음의 ref 확인, 제품 검수를 각각의 기록으로 구분.

리뷰 승인, 로컬 커밋, 원격 전달, 제품 검수 완료는 서로 다른 결과다. 계획 작업은 승인 계획과 코드를 연결하고, 일반 작업도 리뷰한 변경과 전달할 변경을 대조한다. 사용자가 검수 범위를 좁히면 나머지 범위를 완료로 표시하지 않는다.




## 로컬 완료와 원격 반영

리뷰한 변경을 커밋한 뒤 부모/트리/변경 내용을 확인한다. 푸시 응답을 잃으면 원격 OID와 실행 기록부터 대조한다. 커밋과 푸시 성공은 앱 전체의 검수를 대신하지 않는다. 인도 대기에서도 결정문과 함께 같은 구현/리뷰 세션으로 재진입할 수 있다.

## 묶음의 전달 확인 기록

묶음은 전달 대상 브랜치를 선택적으로 지정할 수 있다. 전달 확인 API는 동결한 통합 결과 커밋이 로컬 원격 추적 ref에 포함됐는지 검사하고, 확인한 ref/SHA/시각과 포함 여부를 남긴다. ref가 없거나 결과가 미포함인 경우도 기록한다.

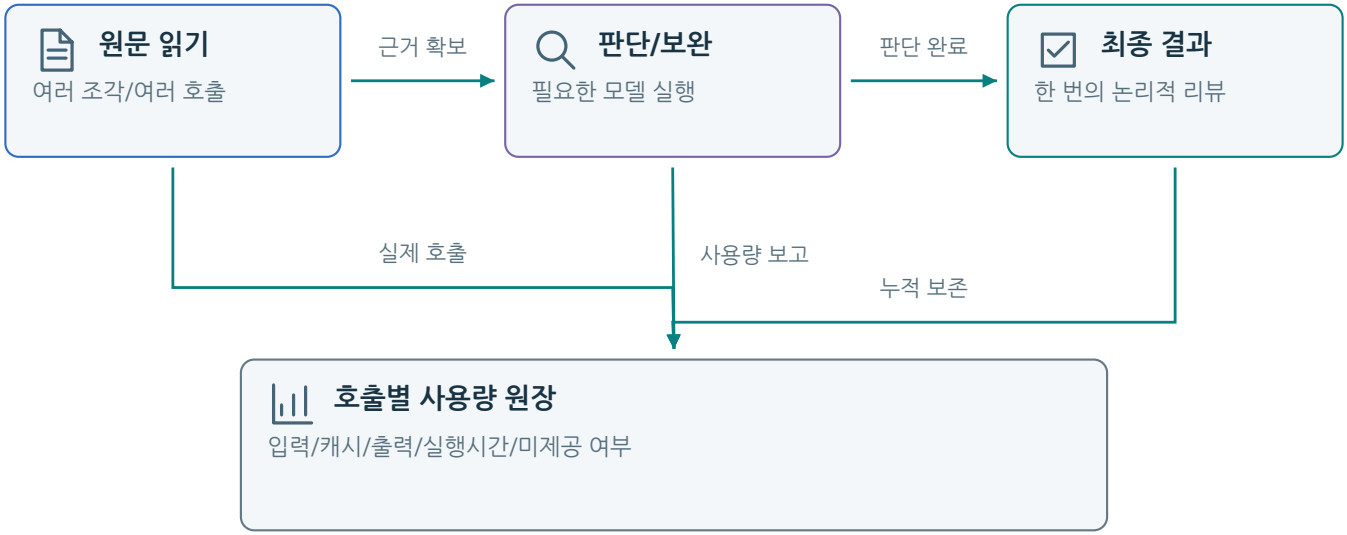
이 API는 fetch/push/merge를 실행하지 않는다. 원격 최신화는 호출 전에 중재자가 수행한다. 확인 기록만으로 묶음을 닫거나 전달 완료 판정과 인계 알림을 바꾸지 않는다.

src/server/engine/delivery.ts / workGroupService.ts / src/shared/workGroups.ts

# 사용량 기록과 실행 한도

원문 읽기 횟수, 논리적 리뷰, 실제 모델 실행을 서로 다른 값으로 기록.

대화 왕복 수, 리뷰 회차, 실제 모델 호출 수는 같지 않다. 형식 교정이나 공급자 재시도가 있으면 한 역할 턴에도 여러 호출이 생길 수 있다. 입력/캐시 입력/출력/시간을 호출별로 기록하고, 작업의 한도와 함께 확인한다.




## 역할별 원문과 실제 실행 횟수

report-usage.py는 운영 DB와 세션 원문을 읽어 역할별 JSON/Markdown 보고서를 만든다. 자동으로 연결할 수 없는 세션은 작업과 역할을 명시하고, 같은 요청의 사용량은 한 번만 합산한다. 역할 턴의 리뷰 횟수와 실제 모델 호출 수는 별도로 남긴다.

## 누적 사용량과 적용 한도

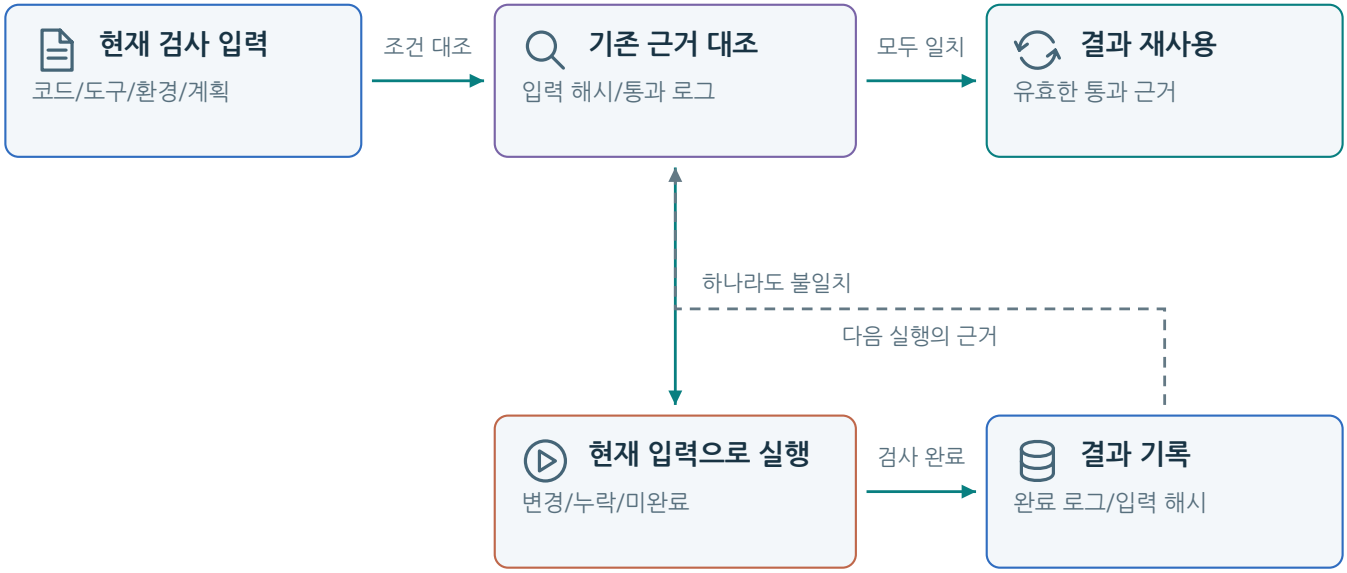
기본 정책은 사용량을 기록하며, 사용자가 정한 예산/횟수와 공급자의 제한은 별도로 적용한다. 새 커밋, 세션 교체, 서버 재시작으로 같은 작업의 사용량을 초기화하지 않는다.

9월 파일럿에서는 당시 예산과 리뷰 회차를 추가했다. 누락된 사용량을 0으로 채우지 않았으며 해당 호출과 응답을 보존했다. 이 비용 기록과 최신 구조의 효과를 같은 측정값으로 합치지 않는다.

# 검사 결과의 재사용 조건

두 검사 프로필의 입력/도구/환경/계획/로그가 같을 때 통과 결과를 재사용.

Swift 동시성 정책 정적 검사와 변경된 Swift 파일의 구문 검사, 두 프로필을 지원한다. 코드/도구/환경/로그를 대조하고, 계획 작업에서는 승인 계획도 확인한다. 계획 없는 일반 작업도 검사 결과를 저장하고 재사용한다.




## 같은 검사를 다시 실행하던 비용

작업 단계만 바뀌었다고 같은 검사를 반복하면 실행 비용이 늘어난다. 관련 입력이 같은 통과 결과는 재사용하고, 변경/새 실패/반증이 생기면 해당 검사를 다시 한다. 미추적 Swift 파일도 입력에 포함한다.

## 항목별 결과와 현재 코드의 대조

계획의 checks 선언은 모든 블록을 읽고 중복 ID를 거부한다. 일부 항목만 실패했는데 전체를 실패로 기록하던 문제도 고쳤다. 상태 전이는 집계로 정하고, 각 항목의 통과/실패/실행 불가를 따로 남긴다.

리뷰 재개에서도 현재 코드로 검사 조건을 확인한다. 검사 중 HEAD가 바뀌면 모델을 부르기 전에 거부한다. 정적 검사 통과는 앱 빌드/화면/API 성공의 근거로 확장하지 않는다.

src/server/planCheckGate.ts / src/shared/planChecks.ts / src/server/verifications.ts

# 로그로 확인한 처리 속도

같은 DB 사본의 원문 목록 계산: 25.042초 → 1.072초, 소요 시간 95.7% 감소.

### 원문 186개에 대한 managed 계산




**DB 읽기 97.9% 감소**

8,643회 → 185회
같은 목록 계산 전체의 호출 수

**조회 대상 계산 97.7% 감소**

due: 2.877초 → 0.067초
DB 읽기 989회 → 1회

## 같은 데이터와 같은 결과로 대조

2026년 10월 15일 운영 DB 사본과 고정 시계로 수정 전후를 재현했다. work_groups 3행, 637,567바이트를 중첩 순회마다 다시 읽던 것이 병목이었다. 한 번의 동기 계산 안에서만 재사용하고 범위가 끝나면 버리도록 바꿨다. 비교한 9개 결과의 SHA-256은 모두 같았다.

## 모든 경로가 같은 폭으로 빨라지지는 않았다

승인 재사용 조회는 72.154초 → 2.770초로 96.2% 줄었다. 반면 43개 토픽의 전체 상태 계산은 4.998초 → 4.319초로 13.6% 줄어 다른 조회 비용이 남았다. 위 수치는 해당 계산의 재현 시간이며 전체 제품 작업의 속도나 평균 응답 시간은 아니다.

2026.10.15 운영 DB 사본 재현 기록 / src/server/evidence/catalog.ts / tests/evidence-catalog.test.ts

# 프롬프트 전달량을 줄인 과정

같은 내용을 전달하는 바이트 수와 실제 모델 토큰 사용량을 나눠 측정.

### 9월 13일: 패치 표현

48,588B → 38,324B
21.1% 감소
같은 결과와 SHA를 만든 fixture

### 10월 7일: 계획 변경분

77,401B → 48,232B
37.7% 감소
같은 캡처 입력 재계산

### 현재: 긴 결정 파일 + 순번별 색인

고유 본문은 한 번 / 같은 결정은 참조 / 압축 뒤 원문 재열람

## 먼저 줄였던 것은 전달 표현의 크기

줄 단위 패치는 한 사례에서 10,264바이트를 줄였지만 줄 번호를 붙인 입력 1,464바이트가 추가됐다. 10월 7일에는 같은 캡처의 계획 변경분을 29,169바이트 줄였고, 중복 보고 제거로 다른 개정 과제에서 3,409바이트를 덜 보냈다. 서로 다른 입력이므로 절감량을 합산하지 않는다.

## 표현 최적화에서 전달 구조 개선으로

이전 패치 적용기와 계획 전용 패커는 후속 개편에서 제거했다. 현재는 계획 파일과 diff를 보존하고, 긴 사용자 결정을 파일과 색인으로 전달한다. 같은 본문을 반복하지 않아도 출처와 순서는 남는다. 과거 방식의 절감률을 현재 엔진의 효과로 옮겨 쓰지 않는다.

바이트 감소율은 토큰이나 비용 감소율이 아니다. 같은 모델/도구/입력/출력 범위를 맞춘 실제 호출의 전후 사용량으로 따로 확인해야 한다.

2026.09.13 / 10.07 재현 기록 / src/shared/prompts.ts / src/server/engine/delivery.ts

# 토큰 사용량의 비교 범위

2026년 10월 18일 원장을 추출해 중복 집계와 비교 조건을 먼저 확인.

### 계획 출력 기록

44건
9월 13일~10월 8일

### 최종 사용량 기록

565건
사용량 관측 기록

### 관측 완전성

완전 409건
부분 156건

### 같은 조건인지 대조한 뒤 절감률 계산

작업 범위 / 모델·강도 / 재개·압축 / 실제 요청별 사용량

## 실행 로그를 비교한 결과

9월 13일~10월 18일 사용량 관측 566행 중 최종 기록은 565건, 중간 사용량 관측은 1건이었다. 최종 기록 중에서도 156건은 사용량이 부분 관측이었다. 계획 출력 44건 역시 수정 범위와 출력 형식이 달랐다. 전체 실행 비용을 전후 평균으로 나눠 엔진 절감률로 해석할 조건은 갖추지 못했다.

## 줄인 동작과 측정한 효과의 구분

엔진은 같은 세션을 재개하고, 지침 영수증을 재사용하며, 긴 결정의 중복 본문을 줄인다. 검사도 코드/도구/환경이 같을 때 통과 결과를 재사용한다. 이 경로가 있다는 사실과 실제 토큰이 몇 퍼센트 줄었는지는 별개다.

캐시 입력은 전체 입력에 포함되므로 다시 더하지 않는다. 누락량을 0으로 채우거나 재개 세션의 누적량을 매번 이번 호출량으로 합산하지 않는다. 현재 확인한 개선은 처리 시간과 전달 바이트이며, 전체 작업의 시간/토큰 절감률은 같은 조건의 비교 실행이 필요하다.

2026.10.18 운영 원장 읽기 전용 추출 / optimization_metrics / execution_usage / docs/usage-report.md

# 9월 파일럿의 실행 경로

첫 실패는 보존하고, 엔진 중간 수정 없이 새 파일럿에서 다섯 단계와 원격 전달까지 확인.

첫 파일럿은 예산 처리와 계획 확정 경로의 결함으로 중단됐다. 중단 기록을 유지한 채 관련 계약을 수정하고 검토했다. 그 실행을 이어 붙여 성공으로 만들지 않고 새 파일럿에서 전체 흐름을 다시 확인했다.




## 개발 네 단계와 최종 통합

두 번째 파일럿은 개발 네 단계와 최종 통합을 실제 Claude/Codex CLI로 수행했다. 진행 중 엔진 파일을 바꾸지 않았고, 비교 대상 243개 파일의 해시가 유지됐다. 마지막에는 모든 단계를 닫고 최종 통합 결과를 격리된 bare 원격에 전달했다.

## 대상 검사와 실패 검출

최종 대상 프로젝트의 검사 77개가 통과했고, 고의로 필요한 export를 제거한 반대 검사에서는 실패했다. 통과 숫자만으로 판단하지 않고 검사가 실제 결함을 거부하는지도 확인했다.

docs/usage-report.md / 9월 파일럿 당시 관측 결과

# 파일럿 사용량과 검증 범위

9월 파일럿의 사용량과 10월 개편의 검사 결과는 서로 다른 실행의 증거다.

9월 파일럿은 실제 Claude/Codex CLI를 사용해 다섯 단계를 수행하고 격리된 원격에 결과를 전달했다. 아래 수치는 그 실행에서 수집한 관측값이다. 비교 조건이 같은 이전 실행은 없으므로 이 값을 비용이나 시간의 절감률로 해석하지 않는다.

### 입력 토큰 22,602,633



캐시 입력 18,134,381
전체 입력에 포함

그 밖의 입력 4,468,252

출력 620,113 토큰 / 실행 48회 / 호출 시간 합계 6,566초

## 캐시 입력과 실행시간의 해석

캐시 입력은 전체 입력에 포함되며 다시 더하지 않는다. 6,566초는 호출 시간의 합계로 전체 작업의 경과 시간과 다르다. 파일럿은 예산/리뷰 횟수 추가가 있었고, 외부 자료도 저장된 시험용 스냅샷이었다. 완전 무인 실행으로 소개하지 않는다.

## 10월 9일 엔진 검사와 제품 검수

2026년 10월 9일 흐름 단순화 후보의 실행 기록에서 엔진 검사 1,686개와 호스트 리뷰 검사 275개 통과를 확인했다. 조건부 8개는 건너뛰었고 TypeScript/웹 빌드/공식 리뷰를 확인했다. 코드와 검사 입력이 같은 통과 결과는 재사용했다.

대상 iOS 앱은 매물 등록 2단계의 화면/입력/로컬 검증/임시저장 복원을 검수했다. 관련 검사 980개와 디자인 시스템 179개가 통과했고, 같은 CR 리뷰어가 추가 지적 없이 승인했다. 다른 단계와 매물 전송 API 성공은 이 승인에 포함하지 않는다.

docs/usage-report.md / 9월 파일럿과 10월 개편/제품 검수는 별도 실행

# 실제 iOS 앱의 Swift 6 전환

2026.08.30 - 09.23, 약 4주에 걸친 전환. 10월 개편 전 엔진으로 수행한 운영 사례.

운영 중인 iOS 앱을 Swift 6로 전환하면서 Consensus Room을 사용했다. 공통 모듈부터 기능 모듈/앱/테스트까지 의존 순서로 작업을 나누고, 각 단계의 계획/구현/검토 기록을 보존했다.




**각 단계에서 계획 합의 → 승인 → 구현 → 교차 검토 → 검증**

범위와 판단은 사람이 정하고, 두 모델의 응답/지적/수정 결과를 같은 작업에 보존

## 직접 맡은 판단과 모델별 역할

작업 범위와 설계 판단, 계획 승인은 직접 맡았다. Claude의 구현을 Codex가 교차 검토하고, 지적과 수정 내역을 같은 단계에 남겼다. 예외가 필요한 동시성 처리는 적용 범위와 근거를 기록해 다음 단계에서도 확인했다.

## 검토가 실제 수정을 만든 사례

공통 모듈의 전환 전후 비교에서 프로젝트 생성이 실패했는데도 기존 Swift 6 프로젝트로 두 빌드를 수행한 문제가 드러났다. 생성 성공과 실제 컴파일 언어 모드를 먼저 확인하도록 절차를 고치고, Swift 5와 Swift 6의 비교 결과를 다시 수집했다.

## 전환 완료와 재유입 방지

9월 23일 전환 코드를 개발 브랜치에 통합하고, 앱/테스트를 포함한 프로젝트의 Swift 언어 모드 기본값을 6.0으로 올렸다. 전환 후에는 Swift 5 설정이 다시 들어오면 실패하도록 CI 검사를 추가했다. 구현 완료뿐 아니라 후속 변경에서도 전환 결과를 지킬 수 있게 했다.

단계별 계획/구현/검토 기록 / iOS 전환 커밋 / Project.swift / ci/check_swift_version_policy.py

# 기존 작업을 보존하는 재개

저장된 결과와 현재 작업 트리를 대조한 뒤, 허용된 지점에서 이어 간다.

기존 정지 작업을 새 구조로 옮길 때 계획/미커밋 코드/세션/사용량을 버리면 복구가 아니다. 저장 상태를 먼저 이행하고, 현재 상태와 재개 지점을 확인한 뒤 허용된 흐름으로 잇는다.




## 재시작과 작업 재개의 구분

옛 상태를 현재 스키마로 먼저 읽던 순서를 고쳐 재개 지점을 보존했다. 재시작은 사용자 중지를 해제하지 않는다. 별도로 승인한 자동 진행 예약이 있으면 현재 권한과 실행 상태를 대조해 복구한다.

## 서버 밖 커밋의 구현 기준 채택

작업 트리 HEAD와 엔진의 구현 기준이 달라 재개가 막힌 사례가 있었다. 현재는 중재자가 확인한 커밋과 사유를 보내 기준으로 채택하는 공식 행동을 제공한다. 허용된 정지 지점, 토픽 브랜치와 기존 기준의 후손 관계를 확인하며 자동으로 임의 HEAD를 받아들이지 않는다.

## 복구와 제품 완료의 구분

기준을 채택하거나 세션을 교체했다고 제품 검수가 끝난 것은 아니다. 남은 구현/리뷰/검사를 현재 과제에서 이어 간다. 10월 9일에 검수한 매물등록 2단계 결과도 다른 단계나 전송 API 성공으로 확대하지 않는다.

src/server/workflowMigration.ts / workflow.ts / engine/continuation.ts

# 오류 교정과 원문 보존

실패를 숨기지 않고 받은 결과를 보존하며, 같은 원인을 처리하는 지점을 줄였다.

**받은 원문: 먼저 보존**

응답/세션/요청 시점의 커서
정상 완료 판정과 분리

**실행 경계: 조건 확인**

권한/취소/세션/Git HEAD 유지
같은 세션 교정 또는 명시적 재개

## 요청 교정만으로 풀리지 않았던 문제

옛 계획 제어는 잘못된 자료 이름을 오류 응답으로 돌려주고 읽기 위치를 보존했다. 그러나 이런 보완을 늘려도 계획 패커와 인용 검사를 복구하느라 제품 작업이 멈췄다. 새 계획 왕복은 이 전용 제어를 제거하고 같은 세션이 파일을 읽고 응답하도록 바꿨다.

## 수신 결과를 잃지 않는 공통 처리

결과를 받은 뒤 계획 파일 읽기, Git 조회, 래퍼 후처리가 실패하는 경로를 따로 추적했다. 수신 결과와 그 요청의 입력 순번을 먼저 보존하고, 정상 채택 뒤에는 임시 보존 슬롯을 비운다. 늦은 결과를 두 번 저장하는 회귀도 검사했다.

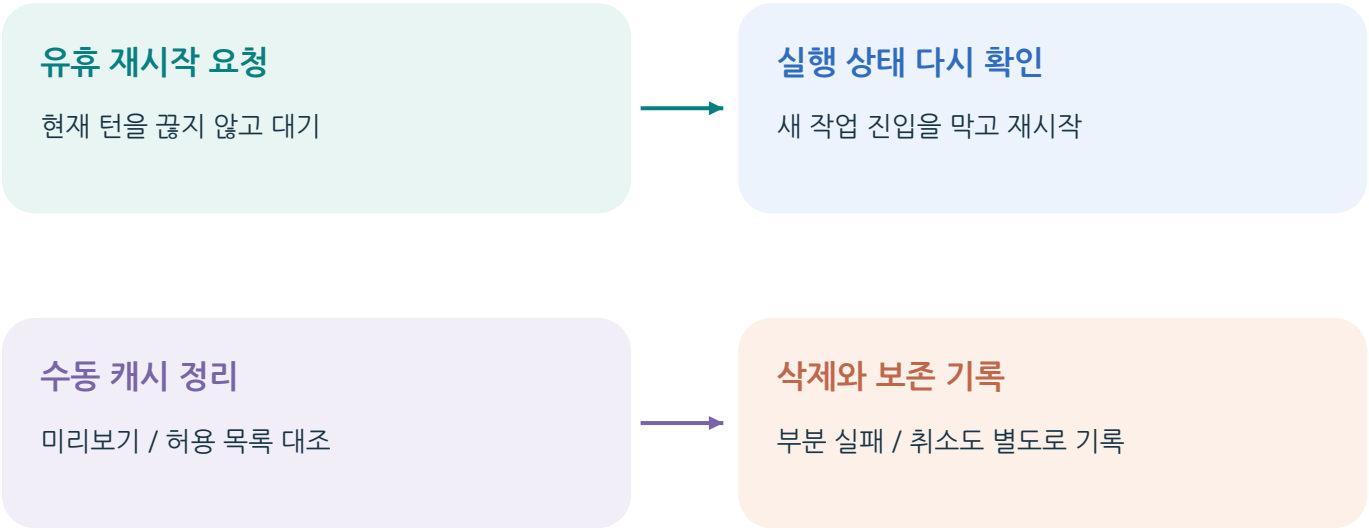
## 남겨야 할 검사와 없애야 할 예외

형식이 틀린 응답은 같은 세션에서 한 번 교정한다. 권한/취소/세션 신원/Git HEAD 검사는 유지한다. 반면 옛 세션 계보에 짝이 있다는 이유로 다른 세션 ID를 받아들이던 예외는 삭제했다. 새 세션은 중재자의 명시적 교체로 연다.

src/server/engine/core.ts / src/server/engine/turnExecutor.ts / src/server/adapters/

# 작업을 보존하는 재시작과 정리

진행 중인 턴을 보존하고, 명시적으로 고른 재생성 가능 캐시만 정리.




## 진행 중인 턴을 끊지 않는 재시작

유휴 대기 재시작은 상한 시간 안에서 실행 종료를 기다린다. 유휴 상태에서 새 작업 진입을 막고 다시 확인한 뒤 서버를 바꾼다. 시간이 넘으면 기존 서버를 그대로 두며, 강제 종료 옵션과 함께 쓰지 않는다.

## 지울 수 있는 캐시만 수동 정리

DerivedData의 Build/Index/ModuleCache 계열과 Tuist/.build만 대상으로 삼는다. 작업 트리 자체, 소스, 로그, 테스트 결과와 잠금은 보존한다. 링크와 활성 작업을 확인하고 실제 삭제 직전에도 보호 상태를 다시 대조한다.

삭제 도중 실패하거나 취소되면 완료한 삭제와 미완료 항목을 나눠 남긴다. 확보량에는 완전히 삭제한 파일 크기만 합산한다. 이 값은 디스크의 실제 여유 공간 증가량과 다를 수 있다.

ops/common/restart_room.sh / src/server/buildTrees.ts / src/server/workflow.ts

# 자동화의 한계와 설계 기준

지원 범위 안에서 작업을 제대로 끝내고, 실패해도 이해하고 이어 갈 수 있는 구조를 목표로 삼았다.

### 중재자: 맥락에 따른 판단

지적의 타당성/우선순위/근거 이연
승인된 범위에서 다음 행동 선택

### 엔진: 실행 조건과 기록

계획/코드 버전/권한/필수 검증 확인
실행 식별/저장된 결과/재개 위치 관리

## 예외를 늘릴수록 제품 작업은 더 늦어졌다

계획 읽기, 지적 분류, 근거 심사, 재확인이 서로의 상태에 의존하면서 엔진 수리가 앱 작업을 막았다. 같은 판단을 모으고 대체된 옛 경로를 삭제했다. 파일 수보다 한 변경의 소비처와 복구 경로를 함께 확인하는 일이 중요했다.

## 엔진 결함은 별도 수리 작업으로

제품 작업자가 보고한 결함은 원장에 등록하고 보존한다. 서버가 Codex를 호출해 자동 수정/리뷰/합류하던 경로는 제거했다. master가 분류하고 Claude 수리 워커에게 배정하며, 제품 작업의 계속 진행 여부와 엔진 수리 일정을 구분한다.

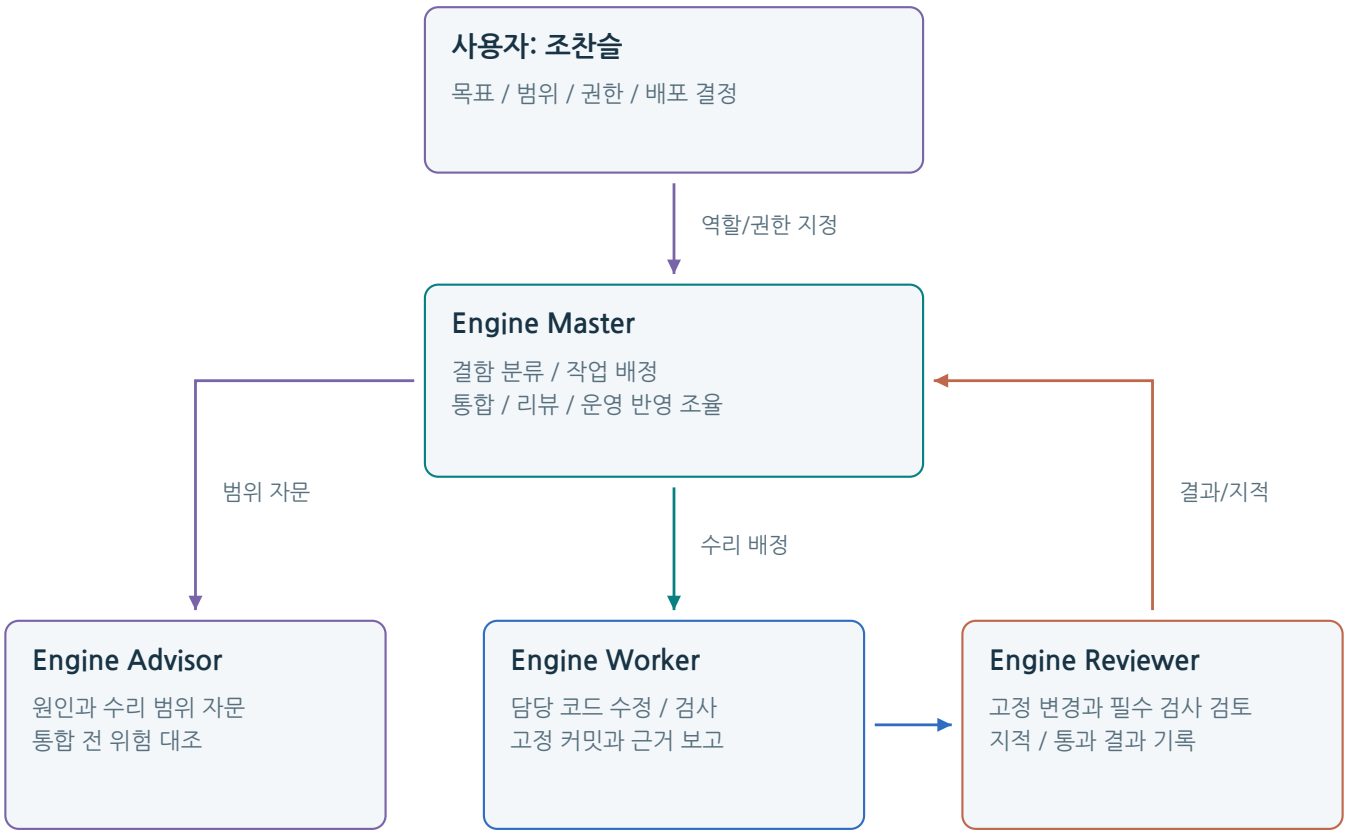
## 판단과 실행, 측정의 구분

작업자와 중재자는 의미와 우선순위를 판단하고, 엔진은 권한/버전/필수 검증/실행 신원/결과를 확인한다. 원문 접근을 제공한 사실을 읽음이나 준수로 해석하지 않는다. 시간과 토큰 효율도 같은 입력의 전후 측정으로 설명하며, 서로 다른 작업의 비용을 절감률로 합치지 않는다.

src/server/engineDefects.ts / engine/core.ts / tools/work-admission/guard.py

# 부록: 네 역할로 나눈 엔진 개발

내가 역할과 권한을 지정하고, 조율 / 구현 / 자문 / 검토를 나눠 엔진을 개선.




## 내가 정한 네 역할

엔진 개선도 한 에이전트에 맡기지 않았다. 내가 목표와 권한을 정하고, Engine Master가 작업을 조율하도록 했다. Engine Worker는 수정과 검사를, Engine Advisor는 원인/범위/통합 전 자문을, Engine Reviewer는 고정된 변경의 검토를 맡도록 지정했다.

## 자문과 검토를 코드 변경에 연결

Master가 실패 근거와 최소 수리 범위를 정리하면 Advisor가 기존 복구로 풀 수 있는지 대조한다. Worker는 담당 작업 트리에서 수정하고 검사와 커밋을 보고한다. Master는 자문을 반영해 통합을 조율하고, Reviewer가 고정한 변경을 검토한다. 지적은 Worker의 수정과 후속 리뷰로 잇는다.

## 제품 작업과 별도로 관리한 엔진 수리

제품 토픽의 플래너/구현자/리뷰어와 별도의 운영 역할이다. 가능한 제품 작업은 계속 진행하고, 엔진 수리의 자문/리뷰/배포와 제품 검수는 각각 확인했다.

사용자 역할 지정 / 엔진 master·worker 보고 / advisor 자문 / 고정 커밋 리뷰 기록